

android

- [android](#)
 - [características](#)
 - [Android Studio vs. IntelliJ IDEA](#)
 - [crear proyecto](#)
 - [descarga](#)
 - [Ejemplo](#)

Android Studio es el entorno de desarrollo integrado oficial para crear aplicaciones Android . Reúne todas las herramientas que un desarrollador de apps necesita en un solo lugar. Android Studio está basado en el potente software IntelliJ IDEA de JetBrains, pero ha sido modificado y mejorado por Google específicamente para el desarrollo de Android . Fue anunciado en 2013 y lanzado oficialmente en 2014, reemplazando a Eclipse como el IDE oficial para este propósito

características

- Editor Inteligente: Soporta de forma nativa Kotlin, Java y C++ . Ofrece autocompletado de código, detección de errores en tiempo real y refactorización (renombrar variables, mover métodos, etc.) de forma segura y automática.
- Sistema de Compilación Flexible (Gradle): El corazón de la construcción de proyectos. Gradle te permite crear diferentes "variantes" de tu app desde un solo proyecto, por ejemplo, una versión gratuita con anuncios y una versión de pago sin ellos, personalizando cada aspecto del proceso de compilación . Los archivos de configuración se llaman build.gradle.kts (usando Kotlin, que es el recomendado) o build.gradle (con Groovy) .
- Emulador Rápido y Potente: Olvídate de necesitar un teléfono físico para cada prueba. El emulador de Android Studio te permite crear dispositivos virtuales de prácticamente cualquier modelo, tamaño de pantalla y versión de Android (teléfonos, tablets, relojes, TVs) para probar tus apps directamente en tu computadora .
- Diseño de Interfaces (Jetpack Compose): Para crear las pantallas de tu app, Android Studio ofrece herramientas visuales que te permiten arrastrar y soltar componentes (botones, textos, imágenes) y ver los cambios reflejados en tiempo real, sin necesidad de compilar todo el proyecto de nuevo .
- Perfiladores y Herramientas de Depuración: Incluye analizadores en tiempo real del uso de CPU, memoria, red y batería de tu app. Puedes pausar la ejecución, inspeccionar el valor de las variables, y encontrar "cuellos de botella" o fugas de memoria que afectarían el rendimiento.

Android Studio vs. IntelliJ IDEA

Característica	Android Studio	IntelliJ IDEA (Community)
Enfoque Principal	Desarrollo de apps para Android (móviles, tablets, wearables, TV) .	Desarrollo general para JVM (Kotlin, Java, backend, aplicaciones de escritorio).
Creador / Soporte	Desarrollado por Google (sobre la base de IntelliJ) .	Desarrollado por JetBrains (los creadores de Kotlin).
Herramientas Exclusivas	Emulador de Android, editores de diseño de layouts (XML y Compose), herramientas	Soporte superior para desarrollo web, frameworks de backend (Spring, Ktor),

	específicas de Google Play (firmado, análisis de APK) .	bases de datos y herramientas de diagramación UML.
--	---	--

¿Cuál elegir? Es la elección obligatoria si quieres crear apps para Android . Es la mejor opción si quieres aprender Kotlin para backend, scripts de consola o aplicaciones de escritorio multiplataforma . En resumen, todo lo que puedes hacer en Android Studio está basado en IntelliJ IDEA . Son "hermanos", pero cada uno está especializado en un tipo de proyecto.

crear proyecto

Creación de un Proyecto (Paso Rápido)

El proceso es muy similar al de IntelliJ que ya conoces, con pequeñas variaciones propias de Android:

1. Al abrir Android Studio, selecciona "New Project" .
2. Elige una plantilla para tu primera pantalla. Se abrirá una ventana con varias opciones. Aquí debes seleccionar la pestaña "Phone and Tablet" y luego elegir la plantilla "Empty Activity". ¿Por qué esta plantilla? Es la más recomendada para empezar con Compose. Crea un proyecto simple con una sola pantalla y todo lo necesario para que veas tu primer "Hola Mundo" funcionando
3. Configura tu proyecto: En la siguiente pantalla, completa los detalles de tu app. Los más importantes son:
 - Name: El nombre de tu aplicación (por ejemplo, "MiPrimeraAppCompose").
 - Package name: El identificador único de tu app (suele ser algo como com.tunombre.miprimeraappcompose).
 - Save location: La carpeta donde se guardará el proyecto.
 - Language: Asegúrate de que esté seleccionado Kotlin. Es el único lenguaje compatible con Compose.
 - Minimum SDK: Selecciona API 24: Android 7.0 (Nougat) o una versión posterior. Esto define la versión más antigua de Android en la que funcionará tu app.
 - Use Jetpack Compose UI: Asegúrate de que esta casilla esté marcada. ¡Esto es crucial! Activa todas las configuraciones necesarias para usar Compose.
4. Define el Minimum SDK: esto determina la versión de Android más antigua en la que funcionará tu app (elige la que quieras, por ejemplo, API 24: Android 7.0).
5. Haz clic en "Finish". Android Studio generará toda la estructura de carpetas y archivos y sincronizará el proyecto con Gradle automáticamente.

Breve descripción:

Gradle Scripts es un archivo de configuración, si abres **build.gradle.kts** veras, en *plugins* los módulos importados, en *android* la descripción de proyecto.

app/kotlin+java/com.sunombre.suproyecto/ui.theme contiene los estilos de compose.

app/kotlin+java/MainActivity es el código principal.

descarga

[Descargar Android studio](#)

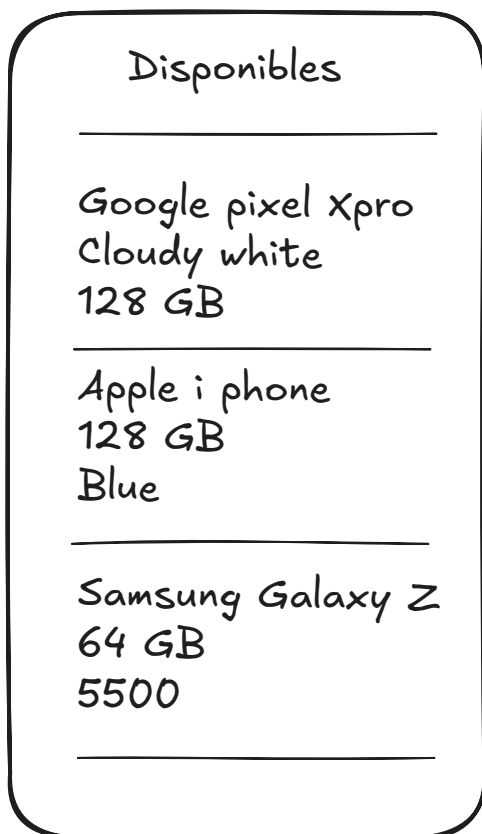
*Nota Requerimientos: memoria 8mb, SO recomendado 64 bits, procesador intel core 2da generacion+. Disco 16 gb+, resolución 1920*1080.*

Ejemplo

Crear una aplicación móvil con Jetpack Compose. Jetpack Compose es el kit de herramientas moderno de Android para construir interfaces de usuario (UI) de forma declarativa utilizando código Kotlin. Fue lanzado por Google para reemplazar el sistema tradicional de vistas basado en XML (como LinearLayout, TextView, Button, etc.). Ejemplo: en lugar de decir cómo construir la UI paso a paso (imperativo), con Compose declaras cómo debe verse la UI según el estado actual de los datos. Antes `findViewById<TextView>(R.id.texto).text = "Hola"` ahora:

```
Text("Hola")
```

Interfaz de la app:



1. Crea un nuevo proyecto
 - file/new project/
 - elige la plantilla `empty activity`
 - elige nombre del **Package name**
 - en **Build configuration language** elegir **kotlin**
 - en **minimum sdk** versión mínima de **Android** donde corra tu app.
 - da clic en `next` .
2. revisa el archivo `/src/MainActivity.kt` es el equivalente al `main()` en lenguaje C.
3. Ubica una api fake: [Api fake](#)
4. elige [all](#)
5. crea una clase para almacenar los datos:
 1. colócate en el paquete principal: **com.sunombre.suproyecto**/clik derecho/new/(kotlin class/file)
 2. elije **data class**
 3. ponle como nombre *device*
 4. ponle las propiedades:

```
data class device(  
    val id: Long,  
    val name: String,  
    val data: spec?  
)
```

5. observa que data hace referencia a spec (especificaciones del objeto como color y capacidad [observa los datos]), por tanto necesitamos crear una nueva class data que llamaremos *spec*, construyela con los coampos:

```
data class spec(  
    val color: String?,  
    val capacity: String?  
)
```

6. código

Observa en el archivo `Main.activity` que tenemos una clase que hereda de `ComponentActivity` Una Activity es un componente fundamental de una aplicación Android que representa una pantalla con la que el usuario puede interactuar.

La función `fun onCreate()` se ejecuta al crear la actividad, dentro puede ver `setContent {}` que es una sentencia del Jetpack Compose que añade la interfaz gráfica a la actividad. `DeviceTheme{}` define el tema, es tomado de Material design.

`scaffold{}` es el contenedor principal de la interfaz.

dentro se llama a la función `Greeting(name = "Android", modifier = Modifier.padding(innerPadding))` Las vistas en Jetpack compose se construyen a partir de funciones kotlin marcadas con `@Composable` y se deben escribir en Pascal Case.

en la función `Greeting` tenemos un componente:

```
Text(  
    text = "Hello $name!",  
    modifier = modifier  
)
```

*Nota Una **Data Class** es una clase especial en Kotlin diseñada exclusivamente para almacenar datos. Automáticamente genera código útil como equals(), hashCode(), toString(), copy(), y los métodos de componente para deestructuración.*

Nota Un componente de Android es un bloque de construcción fundamental de una aplicación Android. Cada componente tiene un propósito específico y un ciclo de vida propio, y pueden activarse de diferentes maneras (por el usuario, el sistema u otros componentes).